

Programmation Orientée Objet en Java

Séance : 05

ISCAE

Dr. Mohamed Ould Djibril

December 5, 2019

Institut Supérieur de Comptabilité et d'Administration
des Entreprises - ISCAE

Objectifs du cours d'aujourd'hui

Aspects syntaxiques du langage Java

- Définition des variables et des types
- Les expressions
- Les instructions conditionnelles
- Les instructions itératives
- Les tableaux

Les types primitifs

Définition:

On appelle des types simples, ou types primitifs en java :

- ① Les entiers
- ② les réels
- ③ les booléens
- ④ les caractères

Les types primitifs

Le type byte

Le type **byte** (1 octet) peut contenir les entiers entre -128 et $+127$.

```
1 byte temperature;  
2 temperature = 64;
```

Le type short

Le type **short** (2 octets) contient les entiers compris entre -32768 et $+32767$.

```
1 short vitesseMax;  
2 vitesseMax = 32000;
```

Les types primitifs

Le type `int`

Le type `int` (4 octets) va de $-2 * 10^9$ à $2 * 10^9$ (2 et 9 zéros derrière...).

```
1  int temperatureSoleil;  
2  temperatureSoleil = 15600000;
```

Le type `long`

Le type `long` (8 octets) de $-9 * 10^{18}$ à $9 * 10^{18}$.

```
1  long anneeLumiere;  
2  anneeLumiere = 946070000000000000;
```


Les types primitifs

Le type boolean

Le type **boolean** contient **true** (vrai) ou **false** (faux).

```
1 boolean question;  
2 question = true;
```

Le type char

Le type **char** contient UN caractère stocké entre de simples quotes ' ', exemple:

```
1 char caractere;  
2 caractere = 'A';
```

Les types primitifs

Manipulations pratiques

- On peut très bien compacter la phase de déclaration et d'initialisation en une seule phase ! Comme ceci :

```
int entier = 32;  
float pi = 3.1416f;  
char caractere = 'z';  
String mot = new String("Coucou");
```


Les types primitifs

Manipulations pratiques

- Lorsque nous avons plusieurs variables d'un même type, nous pouvons compacter tout ceci en une déclaration comme suit :

```
int nbre1 = 2, nbre2 = 3, nbre3 = 0;  
char a, b = '0';
```

- On peut aussi convertir une variable ou une formule dans un autre type de donnée via ce qu'on appelle un **cast** :

```
int i = 123;  
double j = (double)i;
```

Les types primitifs

Manipulations pratiques

- Il peut y avoir des pertes de précision au sein même des opérations mathématiques que la conversion du résultat via un cast n'empêchera pas. Ex :

```
int i = 123, j = 5;  
double k = (double) (i / j);
```

Les types complexes

Les énumérations : le type **enum**

- Introduit depuis Java 5.0
- Permet de définir un ensemble fini de valeurs d'une donnée;
- Par exemple : pour définir les valeurs possibles qui vont caractériser l'état de cette donnée.
- **Une énumération est un ensemble fini d'éléments constants.**

```
public enum Jour {  
    LUNDI, MARDI, MERCREDI, JEUDI,  
    VENDREDI, SAMEDI, DIMANCHE  
}
```

Les types complexes

Le type **enum**: Définition

La définition d'une énumération ressemble à celle d'une classe avec quelques différences :

- utilisation du mot clé **enum** introduit spécifiquement dans ce but à la place du mot clé **class**
- un ensemble de valeurs constantes définies au début du corps de la définition, chaque valeur étant séparée par une virgule
- Par convention le nom des constantes est en majuscule
- Une énumération peut prendre plusieurs formes et être **enrichies de fonctionnalités** puisqu'une énumération est une classe Java.

Les types complexes

Le type **enum** : exemples

```
public class TestEnum2 {  
    public enum MonStyle {  
        STYLE_1, STYLE_2, STYLE_3, STYLE_4, STYLE_5  
    };  
  
    public static void main(String[] args) {  
        afficher(MonStyle.STYLE_2);  
    }  
  
    public static void afficher(MonStyle style) {  
        System.out.println(style);  
    }  
}
```

Les types complexes

Le type **enum** : exemples

```
enum Animal { KANGOUROU, TIGRE, CHIEN, SERPENT, CHAT };  
class Test{  
    public static void main(String[] args){  
        String aniMsg;  
        Animal bebetes = Animal.TIGRE;  
        switch(bebetes) {  
            case KANGOUROU :  
                aniMsg = "kangourou"; break;  
            case TIGRE :  
                aniMsg = "tigre"; break;  
            ....  
            case CHIEN :  
                aniMsg = "chien"; break;  
        }  
        System.out.println("L'animal est un "+aniMsg);  
    }  
}
```

Les types complexes

String : Un type complexe particulier

- Le type **String** correspond au type chaîne de caractères.
- Il ne s'agit pas d'une variable mais d'un objet qui instancie une classe qui existe dans Java.
- nous pouvons l'initialiser en utilisant l'opérateur unaire **new()**.

```
String phrase;  
phrase = "Titi ou Tata";  
//Deuxieme methode de declaration de type String  
String str = new String();  
str = "Une autre chaine de caracteres";  
//La troisieme  
String string = "Une autre chaine";  
//Et une quatrieme pour la route  
String chaine = new String("Et une de plus !");
```

Les types complexes

String : Quelques méthodes

String étant un **objet**, il possède des méthodes afin de les manipuler. En voici quelques exemples :

- **toLowerCase()**

Cette méthode permet de transformer tout caractère de la chaîne en minuscules.

```
String chaine = new String("COUCOU TouT LE MONDE");  
String chaine2 = new String();  
chaine2 = chaine.toLowerCase();//donne "coucou tout  
le monde"
```


Les types complexes

String : Quelques méthodes

- **concat()**: Cette méthode permet de concaténer deux chaînes de caractères.

```
String str1 = new String("Coucou ");  
String str2 = new String("toi !");  
String str3 = new String();  
str3 = str1.concat(str2); //donne "Coucou toi !"
```

- **length()**: Permet de donner la longueur d'une chaîne de caractères (en comptant les espaces blancs).

```
String chaine = new String("coucou ! ");  
int longueur = 0;  
longueur = chaine.length(); //donne 9
```

Les types complexes

String : Quelques méthodes

- **charAt()**: Permet de renvoyer le caractère qui se trouve à l'indice i , le premier caractère est d'indice 0.

```
String nbre = new String("1234567");  
char caract = ' ';  
caract = nbre.charAt(4); //renverra le caractere 5
```

- **substring()**: Comme son nom l'indique, elle permet d'extraire une sous-chaîne de caractères

```
String chaine = new String("la paix niche"), ↵  
    chaine2 = new String();  
chaine2 = chaine.substring(3,13); //permet d'↵  
    extraire "paix niche"
```

Les types complexes

String : Comparaison

- **equals()** : Permet de voir si deux chaînes de caractères sont identiques.

```
String str1 = new String("coucou"), str2 = new String("toutou");

if (str1.equals(str2)) // Si les deux chaînes sont identiques
    System.out.println("Les deux chaînes sont identiques !");
else
    System.out.println("Les deux chaînes sont différentes !");
```



Les types complexes

String : ... et pour les méthodes restantes

- String est une classe de java
- Toutes les classes de Java sont bien documentées
- Une seule référence : l'API de Java

Les types complexes

Les types complexes: résumé

- Les **types complexes**, ou **objets** se déclarent un peu différemment des types primitifs. En effet, la déclaration est en fait une instanciation, c'est à dire qu'on crée un objet appartenant à une certaine classe d'objet :

```
Ville bdx = new Ville();
```

Les types complexes

Les types complexes: résumé

- Donc, chaque classe peut servir comme type.
- Soit la classe existe dans les *packages* de Java, j'importe le package concerné et je l'utilise.
- Soit elle n'existe pas, je l'écris alors et je l'utilise.
- Exemple: le type Date.

Les opérateurs

Liste dans l'ordre de priorité.

postfixés	<code>[] . (params) expr++ expr--</code>
unaires	<code>++expr --expr +expr -expr ~ !</code>
création	<code>new (type)expr</code>
mult	<code>* / %</code>
add	<code>+ -</code>
relationel	<code>< > >= <= instanceof</code>
égalité	<code>== !=</code>
et	<code>&&</code>
ou	<code> </code>
affectation	<code>= += -= /= %=</code>

Les opérations ternaires

Syntaxe:

- Trois opérandes (variable ou constante) sont mises en jeu pour affecter des données dans une variable.

```
int x = 10, y = 20;  
int max = (x < y) ? y : x; //Maintenant max vaut 20
```

```
int x = 10;  
String type = (x % 2 == 0) ? "C' est pair" : "C' est  
    est impair"; //Ici type vaut "C' est pair"  
  
x = 9;  
type = (x % 2 == 0) ? "C' est pair" : "C' est  
    impair"; //Ici type vaut "C' est impair"
```


L'instruction if

Syntaxe:

```
if (condition)
{
    // les instructions a executer si la condition est ←
    remplie
}
else
{
    // les instructions a executer si la condition n'←
    est pas remplie
}
```

L'instruction if

Exemple :

```
int i = 10;

if (i < 0)
    System.out.println("Le nombre est negatif");

else
    System.out.println("Le nombre est positif");
```

NB: On n'est pas obligés de mettre les accolades quand il n'y a qu'une seule ligne d'instruction dans la boucle.

L'instruction if

Exemple :

On peut aussi mettre des tests multiples :

```
int i = 0;

if (i < 0)
    System.out.println("Ce nombre est negatif !");

else if (i > 0)
    System.out.println("Ce nombre est positif !!");

else
    System.out.println("Ce nombre est nul !!");
```

L'instruction switch

Syntaxe :

```
switch (exp) {  
  case valeur1 :  
    Liste d'instructions  
    break;  
  case valeur2 :  
    Liste d'instructions  
    break;  
  ...  
  case valeurN :  
    Liste d'instructions  
    break;  
  
  default :  
    Liste d'instructions (si aucune valeur ne correspond a ←  
    exp)  
}
```

L'instruction switch

Règles et précisions

- L'instruction switch ne fonctionne que sur des types simples (pour **exp**) : int, short, char ou byte
- Depuis **J2SE 5.0**, il est possible d'utiliser le type **enum**
- A partir de **Java SE 7.0**, vous pouvez utiliser le type **String**
- N'oubliez pas d'insérer les instructions **break** entre chaque test, leur absence est difficile à détecter!

L'instruction switch

Exemple :

```
int nbre = 5;
switch (nbre){
case 1:
    System.out.println("Ce nombre est petit");
    break;
case 2:
    System.out.println("Ce nombre est moyen ");
    break;
case 3:
    System.out.println("Ce nombre est grand");
    break;
default:
    System.out.println("Ce nombre n'est pas compris ←
        entre 1 et 3");
}
```

Les instructions itératives

Introduction :

Les instructions itératives (ou boucles) permettent de répéter l'exécution d'une ou plusieurs instructions tant qu'une condition est vérifiée et sont:

- La boucle **for**
- La boucle **for** améliorée (Depuis J2SE 5.0)
- La boucle **do while**
- La boucle **while**

La boucle for

Syntaxe:

- La boucle **for** comme en **C** inclut un **initialisateur**, une **condition** et un **compteur**
- Il est possible de faire inclure plusieurs expressions de même genre en le séparant par virgule (,) sauf pour la condition.

```
for (initialisateur; condition; compteur) {  
    Instructions;  
}
```


La boucle for

Exemples:

Utilisation standard :

```
for (int i = 0; i < 10; i++) {  
    doSomething();  
}
```

Plusieurs expressions pour l'initialisation et pour le compteur:

```
// A more complex loop using two variables  
for (int i = 0, j = 9; i < 10; i++, j -= 3) {  
    doSomething();  
}
```

Aucune expression !

```
for (;;) {  
    doSomething();  
}
```

La boucle **for améliorée**

Syntaxe:

- Cette variante est valable depuis **J2SE 5.0**
- Elle permet d'itérer sur un tableau ou une collection.
- Chaque élément du tableau ou de la collection sera retourné et sera accessible dans le bloc de la boucle.
- A chaque itération un élément sera retourner jusqu'à ce qu'il n'y a plus d'élément.

```
for (int i : tabInt) {  
    doSomething(i);  
}
```

La boucle for améliorée

Exemples:

```
String tab[]={ "toto", "titi", "tutu", "tete", "tata" };
for (String str : tab)
    System.out.println(str);
```

```
String tab[][] = { { "toto", "titi", "tutu", "tete" }, { "1",
    "2", "3", "4" } };
int i = 0, j = 0;
for (String sousTab[] : tab) {
    i = 0;
    for (String str : sousTab) {
        System.out.println("Element : " + str);
        System.out.println("Element [" + j + "][" + i + "] ←
            est : " + tab[j][i] + "\n");
        i++;
    }
    j++;
}
```

Les boucles while et do-while

Syntaxe:

while :

```
while (i < 10) {  
    doSomething();  
}
```

do-while

```
// doSomething() sera appele au moins une fois  
do {  
    doSomething();  
} while (i < 10);
```

Interruptions des boucles

Break

sortie de la boucle : sort de la boucle et continue l'exécution des instructions se trouvant après la boucle

Continue

passage à l'itération suivante: interrompt l'itération en cours et passe à l'itération suivante

Étiquette de boucles

break et **continue** peuvent être suivis d'un nom d'**étiquette** qui désigne une boucle englobant la boucle où elles se trouvent (une étiquette ne peut se trouver que devant une boucle)

Etiquette de boucles

Exemple: utilisation d'une étiquette

```
boucleWhile: while (pasFini) {  
    ...  
    for (int i=0; i < n; i++) {  
        ...  
        if (t[i] < 0)  
            continue boucleWhile;  
        ...  
    }  
    ...  
}
```

Les tableaux

Définition:

- Un tableau est une structure de données dont les éléments qui le constituent sont de même type.
- En Java, les tableaux sont considérés comme des objets. Ils sont manipulés par des références.
- les variables de type tableau contiennent des références aux tableaux.
- Les éléments du tableau peuvent être d'un type primitif ou d'un type objet.
- vérification des bornes à l'utilisation.
- La taille du tableau est définie à l'exécution. Elle est fixée une fois pour toute.

Les tableaux

Définition d'un tableau

- Pour définir un tableau sans l'initialiser on fait :

```
int tableauEntier [] = new int [6];  
//ou encore  
int [] tableauEntier2 = new int [6];
```

- La définition d'un tableau initialisé est comme suit:

```
// Syntaxe longue  
int [] nombres = new int [5] {20, 1, 42, 15, 34};  
// Syntaxe abreviee  
int [] nombres2 = {20, 1, 42, 15, 34};
```




Les tableaux

Définition d'un tableau

- La création d'un tableau par l'opérateur **new**:
 - Alloue la mémoire en fonction du type et de la taille
 - Initialise chaque élément du tableau par la valeur par défaut du type de base du tableau: **0** ou **false** ou **null** (selon type de base)

Les tableaux

Définition d'un tableau

- La taille peut être une variable.

```
int n=sc.nextInt(); // lire au clavier l'entier n.  
int tab[] = new int [n];
```

- Après exécution, la taille ne pourra plus être modifiée.
- Par contre la référence tab peut changer. Elle peut par exemple référencer un autre objet de taille différente : par **affectation** ou par **new**.

```
int [] tab1, tab2; // tab1 et tab2 deux tableaux  
tab1=new int [10]; // tab1.length retourne 10  
tab2=new int [15]; // tab1.length retourne 15  
tab1=new int [36]; // tab1.length retourne 36.  
                //tab1 designe un nouveau tableau.  
tab1=tab2 ; /*tab1 et tab2 designe le meme tableau ←  
            reference par tab2. tab1.length retourne 15*/
```

Les tableaux

Exemple:

```
public class ExempleTableau {  
  
    public static void main(String[] args) {  
        // DECLARATION  
        double[] tab;  
        // CREATION d'un tableau de dimension 100  
        tab = new double[100];  
        // AFFECTATION DES ELEMENTS DU TABLEAU  
        for (int i=0; i<tab.length; i++)  
            tab[i] = 1.;  
        // AFFICHAGE  
        for (int i=0; i<tab.length; i++)  
            System.out.print(tab[i]+" ");  
        System.out.println();  
    }  
}
```

Affectation de Tableaux

- l'affectation ne modifie que la référence.
- Elle permet le changement de références.
- Soient **tab1** et **tab2** deux tableaux en faisant **tab1=tab2;**
 - la référence contenue dans **tab2** est affectée à **tab1**.
 - Maintenant **tab1** et **tab2** désignent le même objet tableau qui était initialement référencé par **tab2**.

Attention:

Si on veut copier uniquement les valeurs de tab2 dans tab1, alors dans ce cas on peut faire une copie élément par élément

Copie de tableau

- Méthode *manuelle* : création d'un nouveau tableau puis copie élément par élément.
- Copie par appel de `System.arraycopy()`

```
System.arraycopy(src, srcPos, dest, destPos, nb);
```

- `src` : tableau source
 - `srcPos` : indice de `src` du 1er élément copié
 - `dest` : tableau destination
 - `destPos` : indice de `dst` où sera copié le 1er élément
 - `nb` : nombre d'éléments copiés
- Copie **nb** éléments du tableau **src** (source) à partir de l'indice **srcPos** et les affecte dans le tableau **dest** (destination) à partir de l'indice **destPos**.

Comparaison de 2 tableaux

Pour comparer le contenu de deux tableaux:

- Soit on compare un à un les éléments des deux tableaux.
- On peut aussi utiliser la méthode **equals()** de la classe **Arrays**

```
import java.util.Arrays;
...
int[] tab1, tab2;
tab1=new int [10];
tab2=new int [10];
...
// initialisation de tab1 et tab2
...
boolean b=Arrays.equals(tab1, tab2);
if (b){
    ...
}
...
```

Tableaux d'objets

- Les éléments d'un tableau peuvent être d'un type objet.
- La déclaration d'un tableau d'objets crée uniquement des "cases" pour stocker des références aux objets, mais pas les objets eux-mêmes
- Ces références valent initialement **null**.
- Il faut donc créer les objets avant de les utiliser.

```
Etudiants [] etudiantIG = new Etudiant[100];  
// Chaque element du tableau contient une reference ←  
// vers un objet de type Etudiant  
etudiantIG[0].setCNE(10186);  
// Erreur: Il faut creer l'objet pour pouvoir l'utiliser ←  
.  
etudiantIG[0] = new Etudiant(); // Creation de l'objet ←  
etudiantIG[0].setCode(10186);
```

Les tableaux multi-dimensionnels

Définition:

- On peut définir des tableaux **multi-dimensionnels** :

```
int premiersNombres [][] = { {0,2,4,6,8}, {1,3,5,7,9} };
```

- premiersNombres[0][0]** correspondra au premier élément de la première colonne.
- premiersNombres[1][0]** correspondra au premier élément de la deuxième colonne.
- tableau Multi-dimensionnel = Un tableau des tableaux
- La forme n'est pas forcément rectangulaire comme en C.

Les tableaux multi-dimensionnels

Exemples:

```
int [][] nombres = new int [2][]; //Initialisation de la ←  
    premiere dimension uniquement.  
    nombres[0] = new int [3];  
    nombres[1] = new int [2];
```

```
for (int i = 0; i < monTableau.length; i++) {  
    int element = monTableau[i];  
    // traitement  
}
```

```
int i, j;  
for(i=0; i<matrice.length; i++) {  
    for(j=0; j<matrice[i].length; j++) {  
        //Action sur matrice[i][j]  
    }  
}
```

Les mots clés du langage

Liste des mots clés du langage Java

abstract	double	int	super
boolean	else	interface	switch
break	extends	long	synchronized
byte	final	native	this
case	finally	new	throw
catch	float	package	throws
char	for	private	transient
class	goto	protected	try
const	if	public	void
continue	implements	return	volatile
default	import	short	while
do	instanceof	static	

Ce que j'ai appris aujourd'hui

- la base du langage Java

Cours suivant

① Gestion des exceptions

